Tutorial 7: Hashing

Imagine you're building a contact application that needs to store millions of phone numbers and retrieve them instantly by name. Traditional approaches using sorted arrays would require $O(\log n)$ time for lookups, while unsorted arrays would need $O(n)$ time in the worst case. Hashing offers a revolutionary solution: with the right design, we can achieve $O(1)$ expected time for insertions, deletions, and lookups, regardless of the dataset size.

This tutorial introduces **hashing**, one of the most fundamental and practical techniques in computer science for building efficient data structures. At its core, hashing transforms the problem of searching through potentially vast key spaces into simple array indexing operations. By using carefully designed hash functions, we can map arbitrary keys (strings, numbers, objects) to array indices, enabling near-instantaneous data access.

The power of hashing lies not just in its theoretical elegance, but in its ubiquity across real-world systems. Hash tables form the backbone of:

- **Programming languages**: Python dictionaries, Java HashMap, JavaScript objects
- **Databases**: Index structures for rapid query processing
- **Compilers**: Symbol tables for variable and function lookup
- **Caching systems**: Web browsers, CDNs, and distributed systems
- **Cryptography**: Password storage and digital signatures

However, hashing comes with unique challenges. Unlike binary search trees which guarantee $O(\log n)$ performance, hash tables can degrade to $O(n)$ performance when poorly designed. Understanding collision resolution strategies, load factors, and hash function design becomes crucial for maintaining the promised efficiency.

Learning Outcomes

By the end of this tutorial, you will:

- **Master hash function design**: Understand the principles behind good hash functions, including uniform distribution, determinism, and computational efficiency.
- **Implement hash table operations**: Learn to perform insertions, deletions, and lookups while maintaining optimal performance characteristics.
- **Handle collisions effectively**: Explore and compare collision resolution strategies including separate chaining, linear probing, quadratic probing, and double hashing.
- **Analyze performance trade-offs**: Evaluate average-case vs. worst-case complexity, understand load factors, and recognize when rehashing is necessary.
- **Apply hashing to real problems**: Solve practical algorithmic challenges such as duplicate detection, frequency counting, caching implementations, and set operations.
- **Recognize limitations**: Understand when hashing may not be appropriate and how it compares to alternative data structures.

Hashing Fundamentals

Hash Functions

A **hash function** is the mathematical heart of any hash table. It transforms arbitrary keys from a potentially infinite universe into specific array indices within a bounded table. Think of it as a deterministic "address calculator" that always maps the same key to the same index, enabling direct access to stored data.

The fundamental challenge is achieving **uniform distribution**: ideally, keys should be scattered evenly across all table positions to minimize collisions (multiple keys mapping to the same index). A good hash function exhibits three key properties:

- **Deterministic**: The same key always produces the same hash value
- **Uniform**: Hash values are distributed evenly across the table
- **Efficient**: Computing the hash value takes constant time

Consider the simple modulo hash function $h(k) = k \bmod m$, where m is the table size. While elementary, it demonstrates the core principle: compress a large key space into a manageable array range. The following figure shows this function mapping five keys into a table of size 7, achieving perfect distribution with no collisions.

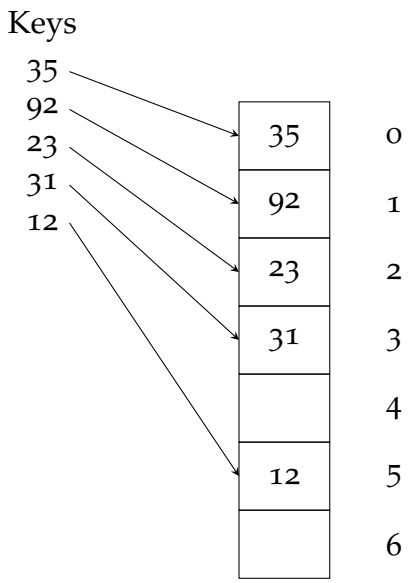

Figure 1: Hashing five keys into a table of size 7 using $h(k) = k \bmod 7$

Collision Resolution

In the real world, perfect hash functions are rare. **Collisions**: multiple keys mapping to the same table index are inevitable due to the pigeonhole principle (when you have more keys than table slots, some slots must contain multiple keys.) The art of hashing lies in handling these collisions gracefully while preserving performance.

Two fundamental approaches have emerged for collision resolution, each with distinct trade-offs:

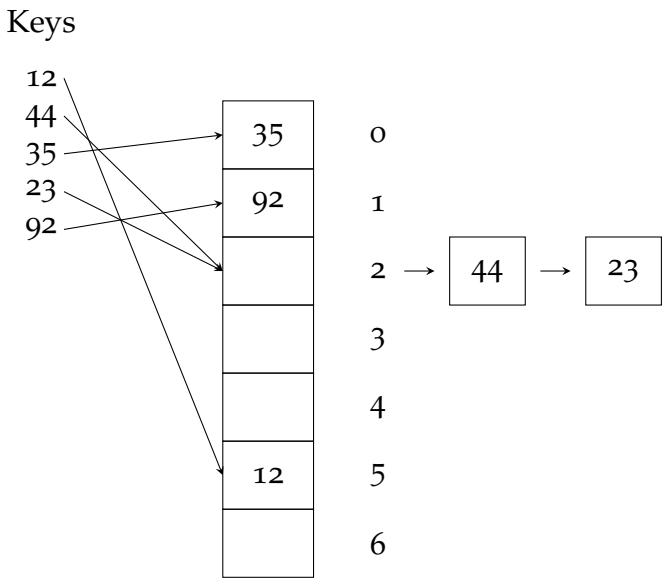

Figure 2: Example: Hashing keys into a table of size 7 using $h(k) = k \bmod 7$. Collisions at index 2 are handled via chaining.

- **Separate Chaining**: Think of each table slot as a "bucket" that can hold multiple items. When keys collide, they form a linked list (or other dynamic structure) at that index. This approach is conceptually simple and handles high load factors gracefully, but requires additional memory for pointers and can suffer from poor cache performance due to non-contiguous memory access.
- **Open Addressing**: All keys live directly within the main table. No external data structures needed. When a collision occurs, the algorithm systematically searches for the next available slot using a predetermined sequence called a **probe sequence**. This approach offers better memory locality and cache performance but becomes inefficient at high load factors.
  - **Linear Probing**: Move to the next slot sequentially: $h(k, i) = (h(k) + i) \bmod m$
  - **Quadratic Probing**: Use quadratic steps to avoid clustering: $h(k, i) = (h(k) + i^2) \bmod m$
  - **Double Hashing**: Use a second hash function for step size: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

Open Addressing: Linear Probing

Linear probing embodies the "try the next slot" strategy: when a collision occurs, simply move to the adjacent slot and check if it's available. If occupied, continue to the next slot, wrapping around to the beginning if necessary. While elegantly simple, this approach creates **primary clustering**. Contiguous blocks of occupied slots that slow down future operations as they require longer probe sequences to traverse.

35	0
92	1
44	2
23	3
	4
12	5
	6

Figure 3: Linear probing: when 23 collides at index 2, it probes to index 3.

Open Addressing: Quadratic Probing

Quadratic probing addresses the clustering problem by using increasingly larger jumps to find empty slots. Instead of checking adjacent positions, it examines slots at quadratic distances: $1^2, 2^2, 3^2, \ldots$ positions away from the original hash location. This approach reduces **primary clustering** (consecutive occupied slots) but can create **secondary clustering** where keys with the same initial hash value follow identical probe sequences.

$$h(k, i) = (h(k) + i^2) \bmod m, \quad i = 0, 1, 2, \ldots$$

The quadratic spacing helps distribute colliding keys more evenly across the table, though it requires careful choice of table size (typically a prime number) to ensure all slots can be reached during probing.

35	0
92	1
44	2
x	3
23	4
12	5
	6

Figure 4: Quadratic probing: 23 collides at index 2, tries 3 (occupied by x), then 4.

Rehashing and Load Factor

The **load factor** $\alpha = n/m$ (where n is the number of stored elements and m is the table size) serves as a critical performance indicator. As α approaches 1.0, collisions become in-

creasingly frequent, and hash table operations slow down dramatically. Think of it like a parking lot: when it's nearly full, finding an empty space takes much longer.

To maintain the promise of constant-time operations, hash tables employ **dynamic resizing** through rehashing:

- **Monitor the load factor**: Most implementations trigger rehashing when α exceeds a threshold (commonly 0.75 for open addressing, or 1.0+ for chaining)
- **Allocate a larger table**: Typically double the size, choosing a prime number to improve hash distribution
- **Rehash all existing elements**: Every stored key must be reinserted using the new table size, as their hash values will likely change

While rehashing requires $O(n)$ time for a single operation, its amortized cost remains constant when performed strategically. This "pay now, save later" approach ensures that hash tables scale gracefully from dozens to millions of elements while preserving their performance guarantees.

Performance Characteristics

Hash tables represent one of computer science's most remarkable performance trade-offs: they sacrifice worst-case guarantees for exceptional average-case behavior. Understanding both aspects is crucial for effective implementation:

- **Average Case Performance**: With a well-designed hash function and controlled load factor, all basic operations (search, insert, delete) achieve $O(1)$ expected time, independent of table size. This makes hash tables invaluable for large-scale applications.
- **Worst Case Scenario**: When all keys hash to the same index (a phenomenon called "hash flooding"), performance degrades to $O(n)$ linear search time. While rare with good hash functions, this vulnerability makes hash tables unsuitable for systems requiring guaranteed response times.
- **Space Complexity**: Hash tables require $O(m)$ space for the underlying array, where $m \geq n$ to maintain reasonable load factors. Additional space is needed for collision resolution structures (linked lists in chaining) or deleted item markers (in open addressing).

The key insight is that hash tables optimize for the common case while accepting rare performance spikes, a design philosophy that works well in most practical scenarios.

Applications

The versatility and performance of hashing have made it indispensable across virtually every area of computing. From the compiler that translates your code to the database serving millions of web requests, hash tables work behind the scenes to make modern computing fast and responsive:

- **Programming Language Internals**: Python's dictionaries, Java's HashMap, and JS's objects all rely on hash tables. These structures enable rapid variable lookup, method dispatch, and dynamic property access that make high-level languages practical.
- **Database Systems**: Hash indexes provide constant-time record retrieval by primary keys, while hash joins efficiently combine data from multiple tables. Modern NoSQL databases like DynamoDB use consistent hashing for distributed data placement.
- **Caching and Memoization**: Web browsers cache resources, CDNs store popular content, and applications memoize expensive computations—all using hash tables to map requests to cached responses in constant time.
- **Data Deduplication**: From identifying duplicate files in backup systems to detecting plagiarism in academic papers, hash tables enable efficient comparison of large datasets by reducing complex objects to simple hash signatures.
- **Security and Cryptography**: Cryptographic hash functions protect passwords, verify data integrity, and enable digital signatures. Hash tables store security tokens, manage user sessions, and implement access control lists.
- **Network Infrastructure**: Internet routing tables use hashing for fast packet forwarding, load balancers distribute traffic across servers, and distributed hash tables enable peer-to-peer networks like BitTorrent.

The ubiquity of hashing reflects a fundamental truth in computer science: most computational problems benefit from fast lookup capabilities. By transforming complex search problems into simple array indexing, hash tables bridge the gap between theoretical elegance and practical performance, making them one of the most valuable tools in any programmer's arsenal.

Warm-up

Problem 1. We have a hash table of size $N > 1$ and a hash function

$$h(k) = k \bmod 2.$$

That means every key is sent either to bucket 0 or bucket 1. We insert a dataset S of n elements, where $n < N$.

Question: After inserting all elements, what is the typical running time of a GET operation (lookup) if we use

- chaining,
- open addressing?

Express your answer as a function of n.

Problem 2. Suppose you are given a hash function h that maps 10-digit integers to integers in the range $\{1, 2, \ldots, 10000\}$. Show that there exists a dataset S of size 1,000,000 such that all keys of S are hashed to the *same* value.

Problem 3. Given two arrays A and B of lengths n and m, respectively, design an algorithm that outputs the set-theoretic intersection $A \cap B$ in expected $O(n+m)$ time and $O(\min\{n, m\})$ extra space.

Problem 4. Work out the details of implementing cycle detection in cuckoo hashing by counting the number of evictions during the insertion process.

Problem 5. Work out the details of implementing cycle detection in cuckoo hashing based on keeping a flag for each entry.

Problem solving

Problem 6. Design a hash table that supports the usual operations (PUT, GET, DELETE) with the additional requirement that when we iterate over the items, they appear in the order in which they were inserted. Iteration over all n items should take $O(n)$ time, and the data structure should add only $O(1)$ overhead to the standard operations.

Problem 7. Given an array of n integers, design an algorithm to find a value that occurs most frequently in the array. Your algorithm should run in $O(n)$ expected time.

Problem 8. A multimap is a data structure that allows multiple values to be associated with the same key. The method GET(k) should return all the values associated with key k. Describe an implementation where this method runs in $O(1+s)$ expected time, where s is the number of values associated with k.

Problem 9. Suppose you have a group of n people and you would like to know if there are two people who share a birthday. Design an $O(1)$-time algorithm that, given the n birthdays, finds a pair with the same birthday or reports that no such pair exists.

Challenge

Intended to stretch your understanding beyond the course material. It is optional and not representative of exam-style questions.

Problem 10. In computational linguistics, texts (such as a book or an article) are modelled as a sequence of words. A *k-gram* is a sequence of k consecutive words. A common task in language modelling is to compute the frequency of all k-grams that appear in the text.

Given a text with n words, design an algorithm that computes the frequency of all k-grams in $O(n)$ expected time.